



Visualizing Complexity

Ways to See the Complexity
in Python Projects

Maxim Danilov

Who I am:



»Maxim Danilov

»Decades of work in Python and Django development

»Working with JS frameworks for the last 7 years.

»Award-winning mentor in engineering

PAPA



Link to slides: github.com/danilovmy/pyconpt2025

Special thanks to my family for the support:

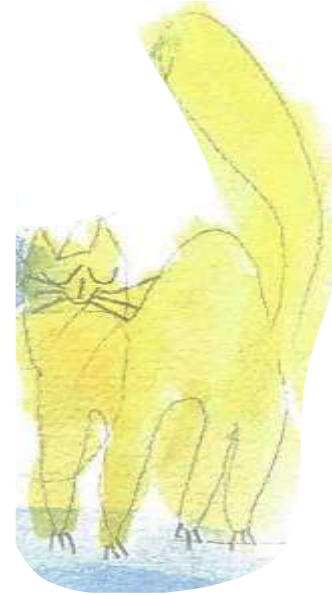


My wife
Elena



daughter
Maja

son
Mark



cat
Kisa



dog
Marselle

Thanks to my children Mark & Maja for the illustrations



7/26/2025

Maxim Danilov maxim@wpshop.at

4

The background is an abstract composition of layered, wavy bands of color. From top to bottom, the colors transition through yellow, pink, purple, and grey, with some darker, more textured areas on the right side. A large, solid blue rectangle is positioned in the center, containing the word 'COMPLEXITY' in white. Below this rectangle is a green rectangle containing the text 'Theoretical recap'.

COMPLEXITY

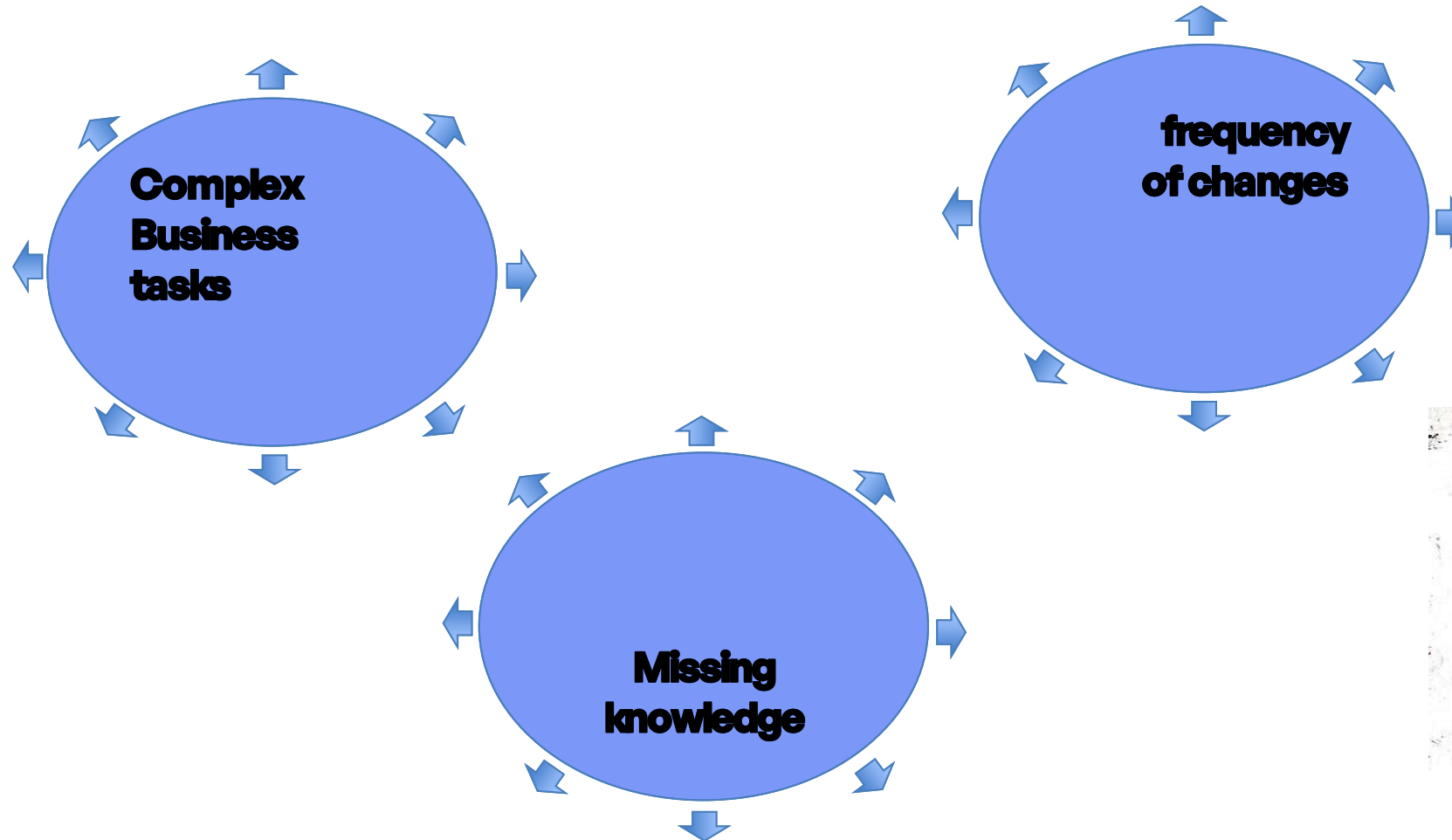
Theoretical recap

Different shadows of Complexity:

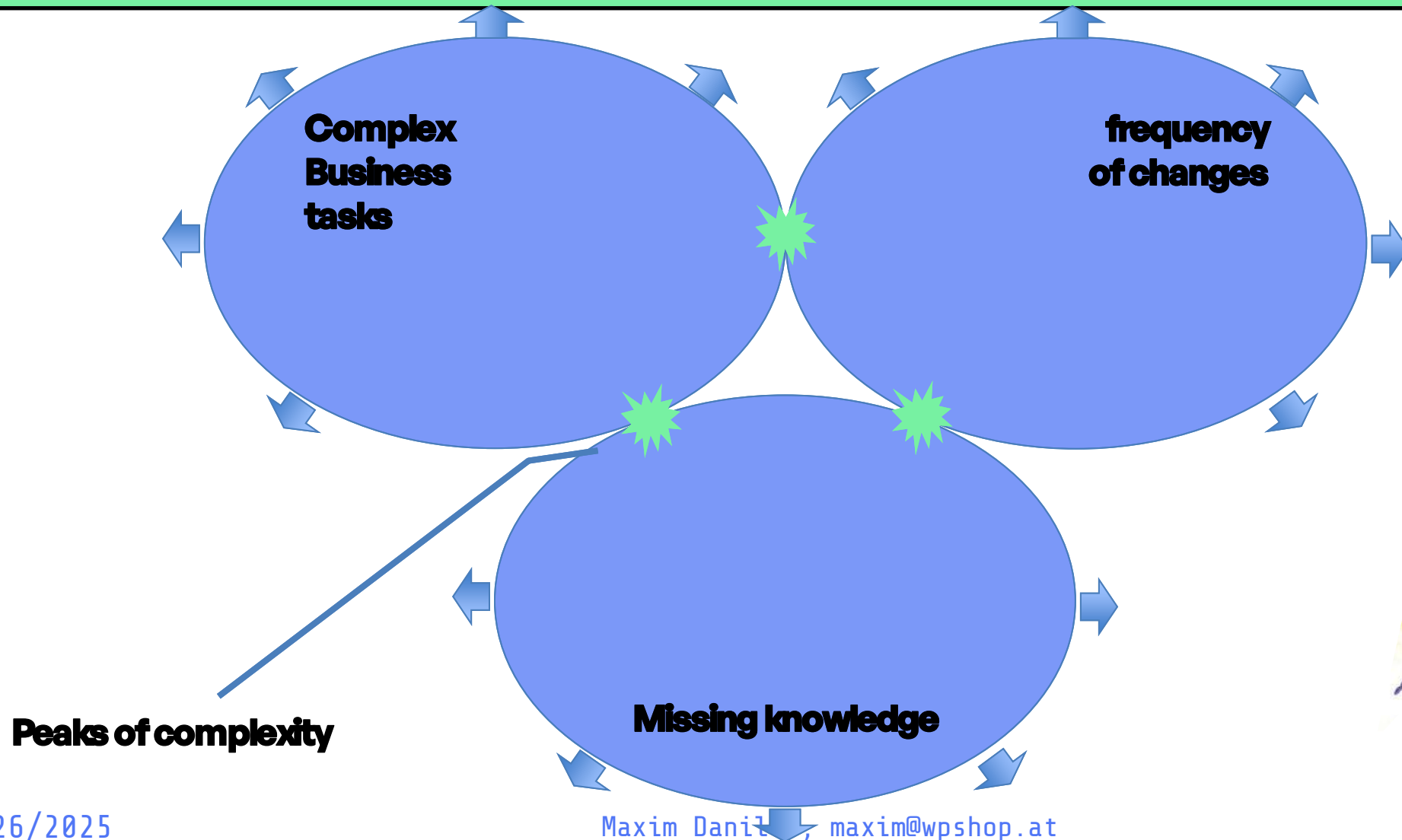
- Structural
 - Grows with relations between objects, removal by changing the coding paradigm
- Cognitive
 - Happens in development as is, reducing by following a strict coding style guide
- Accidental
 - Happens in development due to compromises, removal through refactoring
- Technical (*Essential complexity*)
 - Related to algorithms. Can be frozen, reduced but can not be avoided



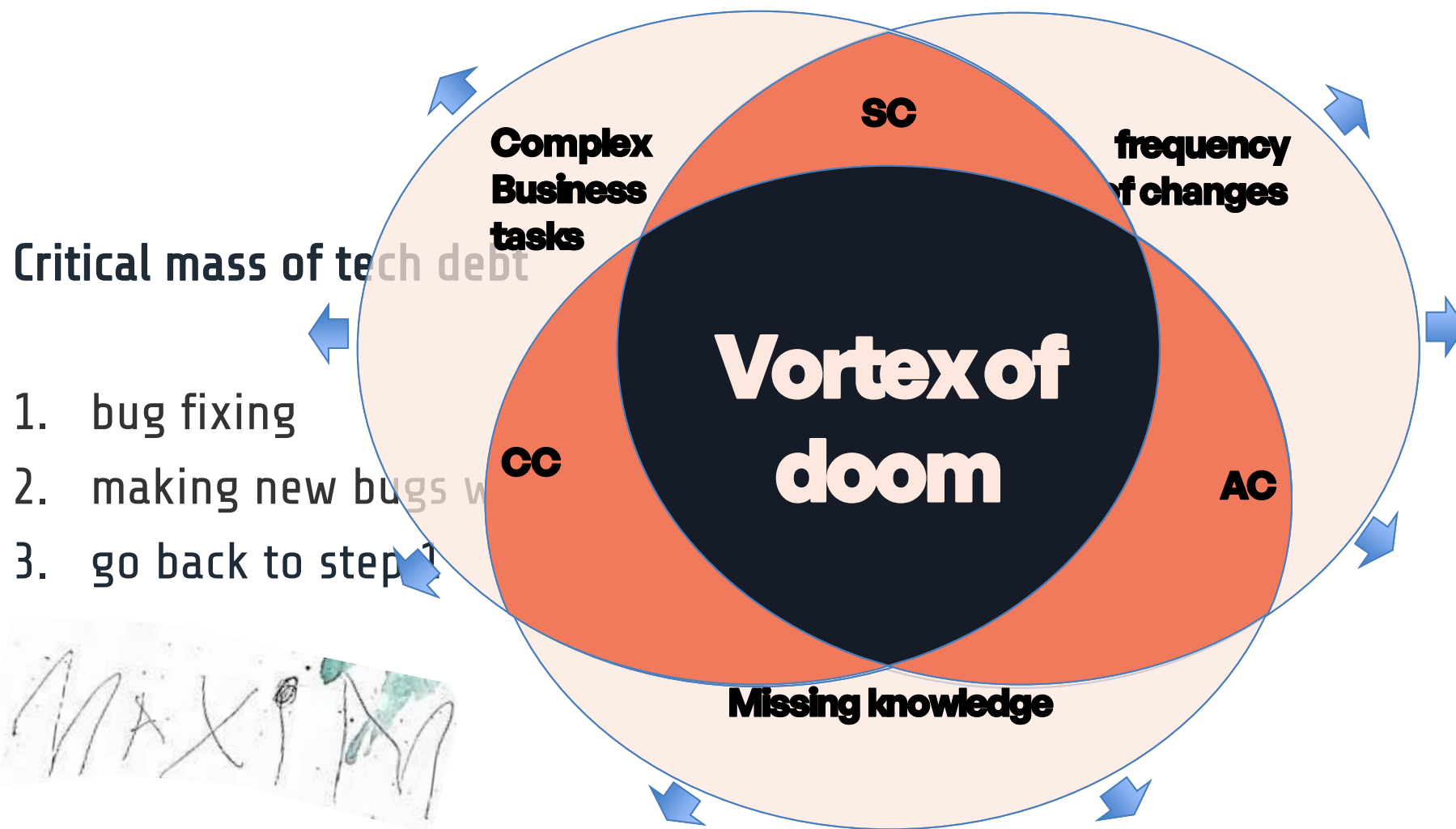
Complexity changes in project life



Matured project



Vortex of doom



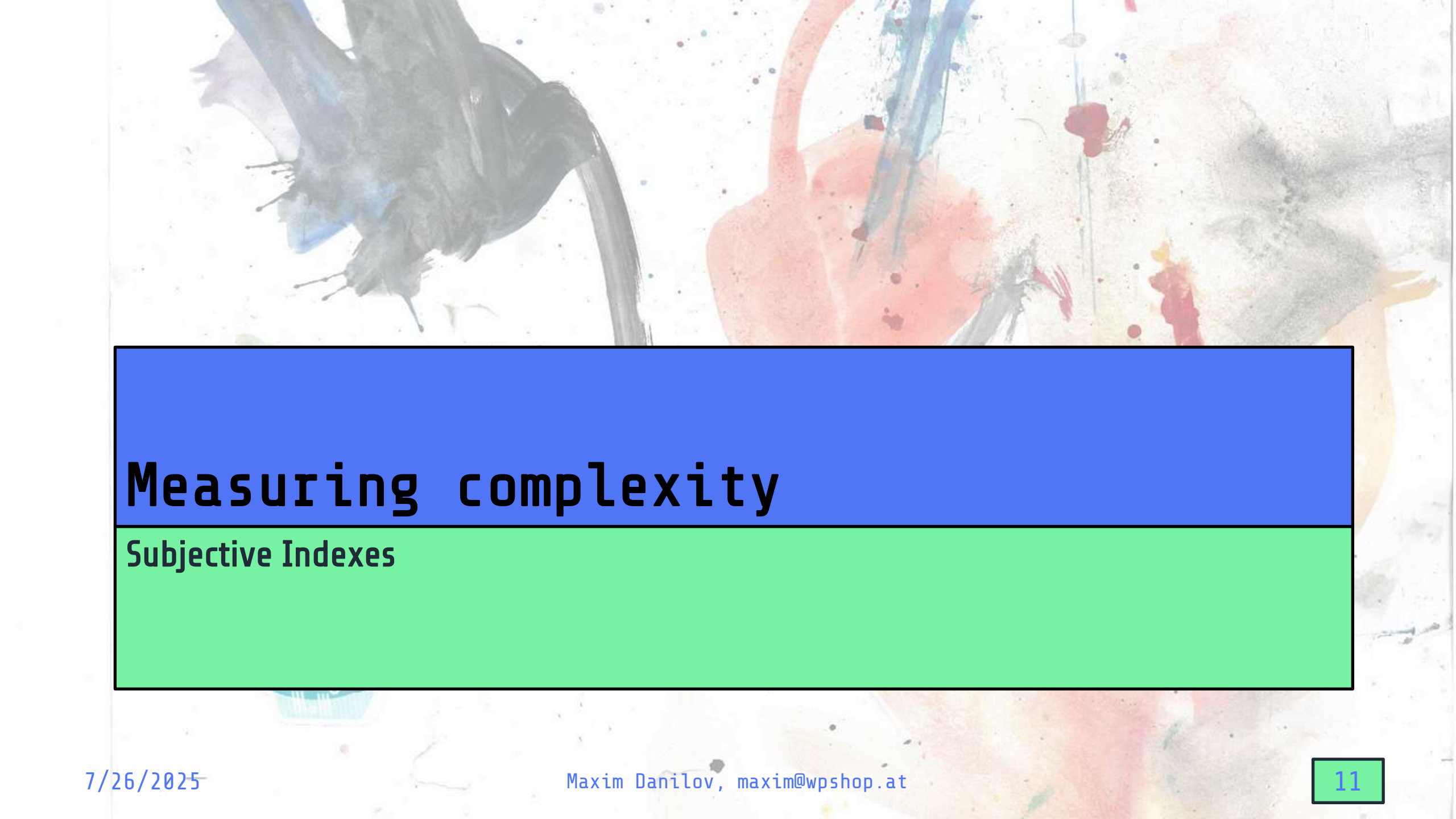
7/26/2025

Maxim Danilov, maxim@wpsshop.at

Complexity theory Summary:

- Complexity invented by humans and subjective.
- Complexity is not the same as a tech debt!
- Complexity is not a bug, but can lead to bugs.
- There is no way to keep complexity at a low level for a long time.
- Managed complexity leads to better design
- Simplicity should be set as KPI





Measuring complexity

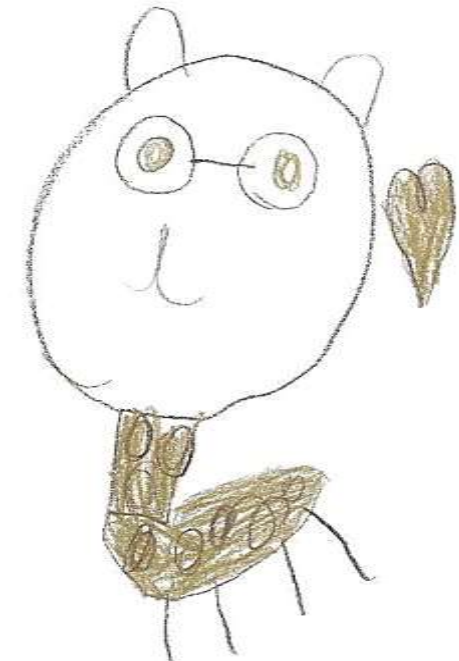
Subjective Indexes

CCL index

- CCL: Count of Code Lines
- Measuring of project size/globality. CC
- Measuring tool: dependency-extractor, perform_extraction -> output.

Cons:

- May be affected by formatters like Black, Ruff, or Prettier.
- Different statements in one line
- Verbose $\neq$ Complex. Long code can be clearer



CCL example

.py files processed: 999

...

total lines of code: 270345

summary lines of code: 47828

max lines in file: 2072

Average: 52 lines/file



class/function index

- OOP vs FB style measurement.
 - + God Functions / God Classes
- Subjective measurement of architecture maturity. CC / SC
- Measured: dependency-extractor, perform_extraction -> output.

Cons:

- Broken by functions composition (FastAPI)
- Pytest/Unittests should not be counted.



class/function example

Commercial project

.py files processed: 999

...

summary count of classes definitions: 1709

summary count of function definitions: 2986

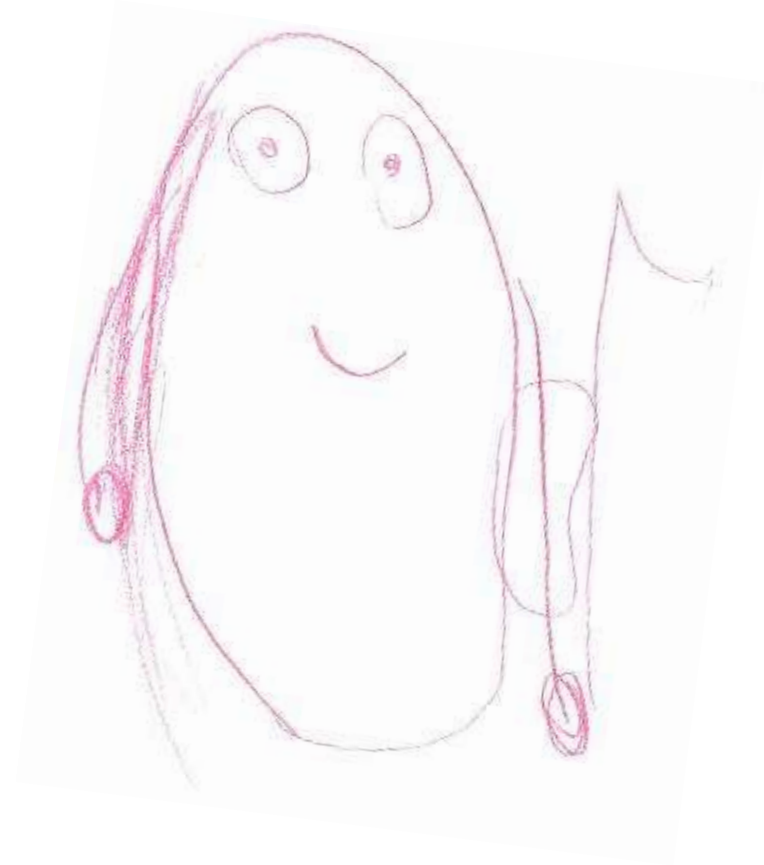
...

36% OOP, 64% FB

average code lines per file: 42

average quantity class/function per file: 5

average length of class/function: 10 lines



Dependency Index

- Coupling/Cohesion index.
- Measuring of structure complexity. SC
- Measured: dependency-extractor, perform_extraction -> output.
- Cons
 - not works with dynamic imports, imports on-fly



Coverage

Coverage report: 99%

Files

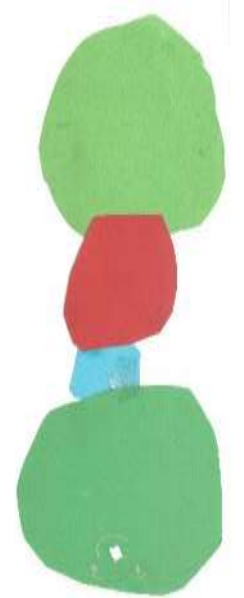
Functions

Classes

coverage.py v7.9.2, created at 2025-07-25 09:55 +0100

File ▲	statements	missing	excluded	coverage
gptests.py	184	1	0	99%
plain.py	83	0	1	100%
Total	267	1	1	100%

coverage.py v7.9.2, created at 2025-07-25 09:55 +0100



Documentation Coverage (CLI)

- docstr-coverage
- Needed: 4; Found: 0; Missing: 4; Coverage: 0.0%
- Overall statistics for 153 files (15 files are empty):
- Needed: 779 - Found: 3 - Missing: 776
- Total coverage: 0.4%
- - Grade: Do you even docstring?



Measuring Complexity

- Cognitive Complexity:
 - Sonar Cloud
- Cyclomatic Complexity:
 - Radon
- Tech Debt metrics:
 - CodeClimate
- Composite code quality index:
 - Codacy



Complexity measurements Summary

- All metrics are subjective!
- Before to do anything – collect your metrics
- After some complexity refinements – collect your metrics
- Dont forget to compare it, to "*confidently make changes in any code base*"



Visualizing complexity

I can see dead peoples...

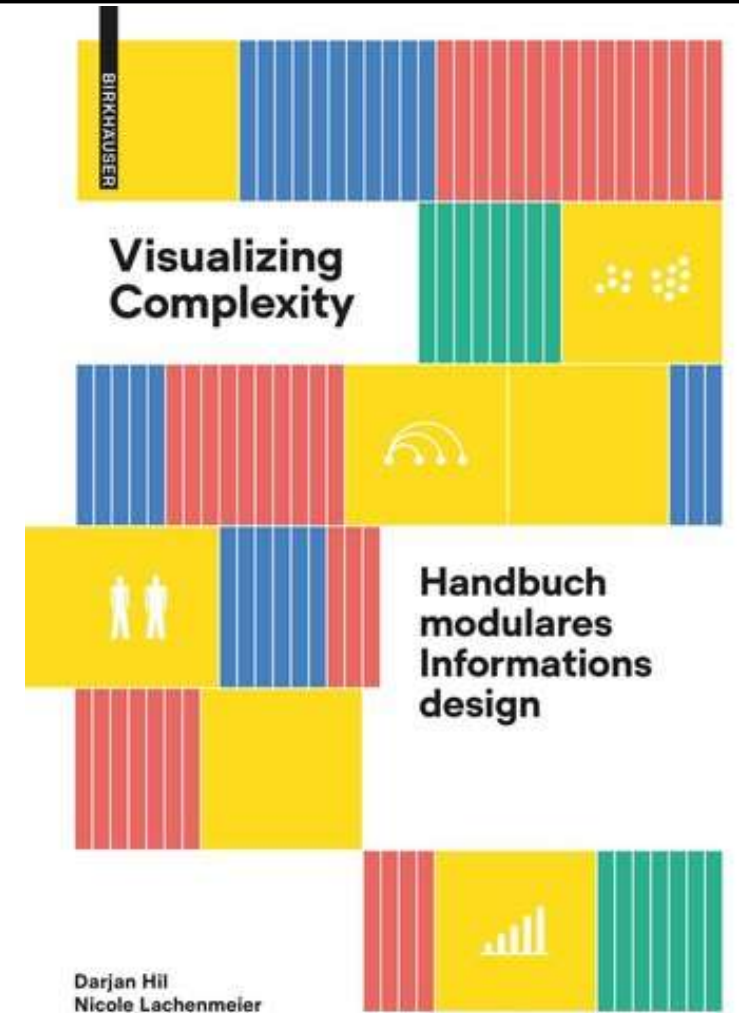
Visualization Theory

- Visualizing Complexity, Darjan Hil Nicole Lachenmeier.

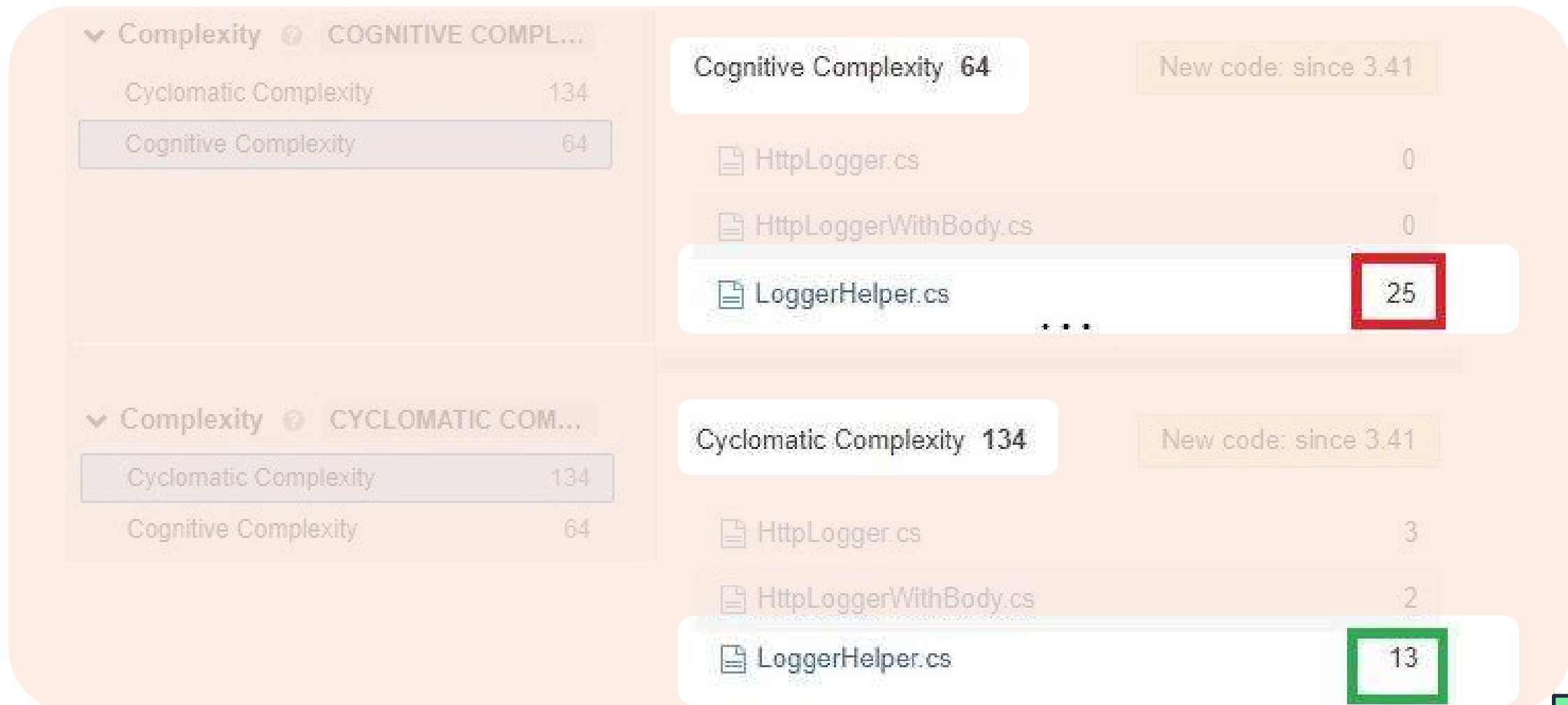
MID's

Modular Information Design System

- Data + Diagrams + Visualization(creativity) + Structure
- Multidimensional visualization

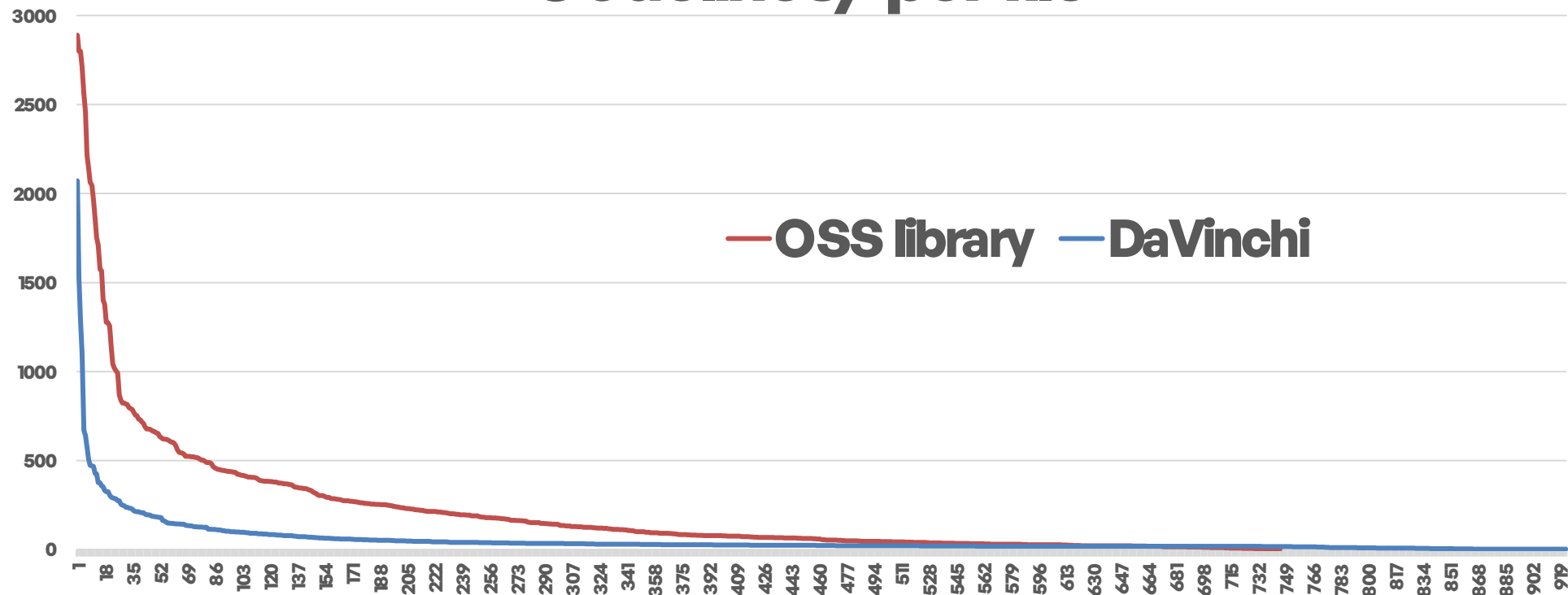


Cognitive / Cyclomatic complexity (Sonar Cloud)



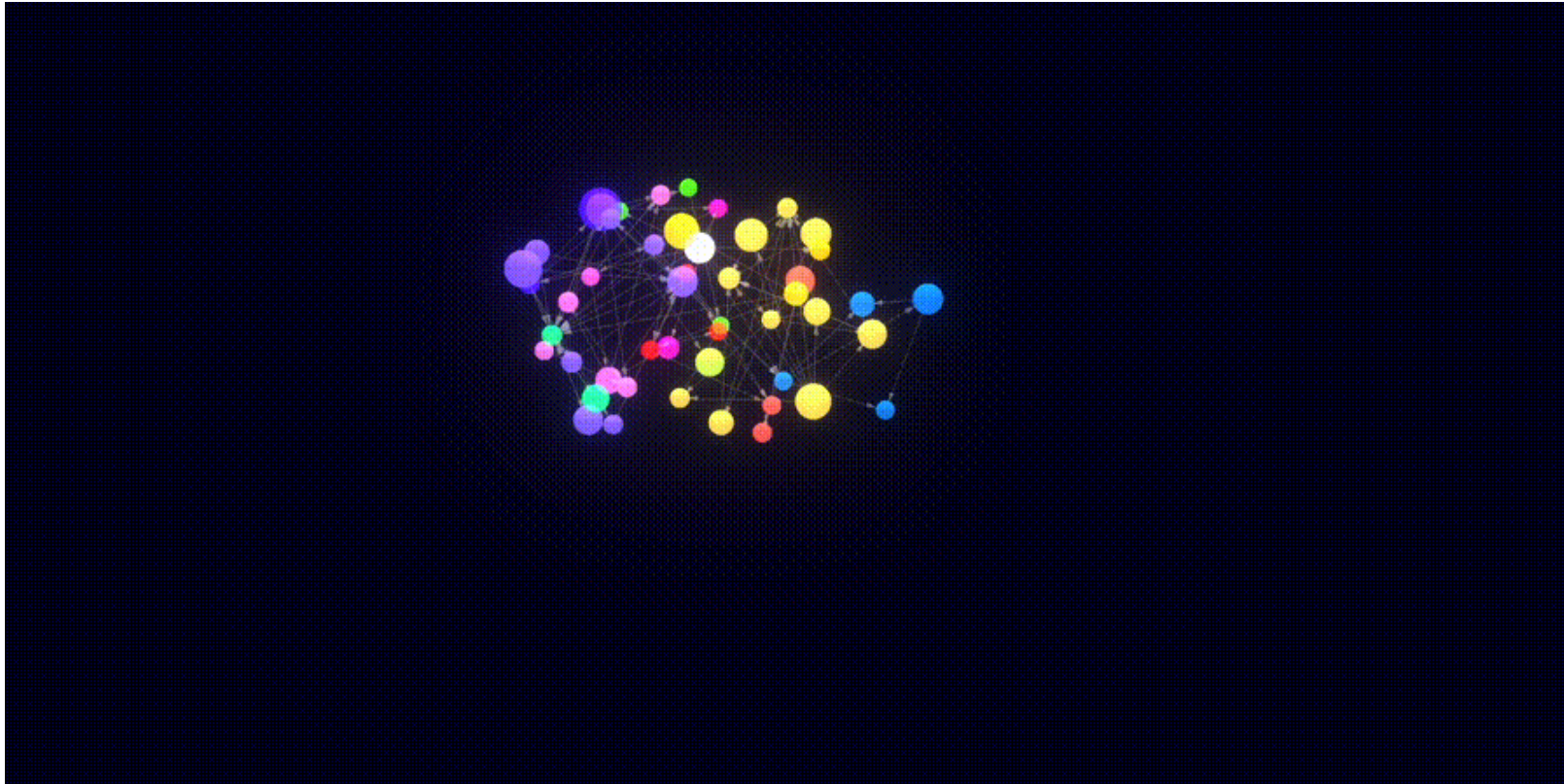
CCL example in xls Diagrams (cc)

Codelines/ per file



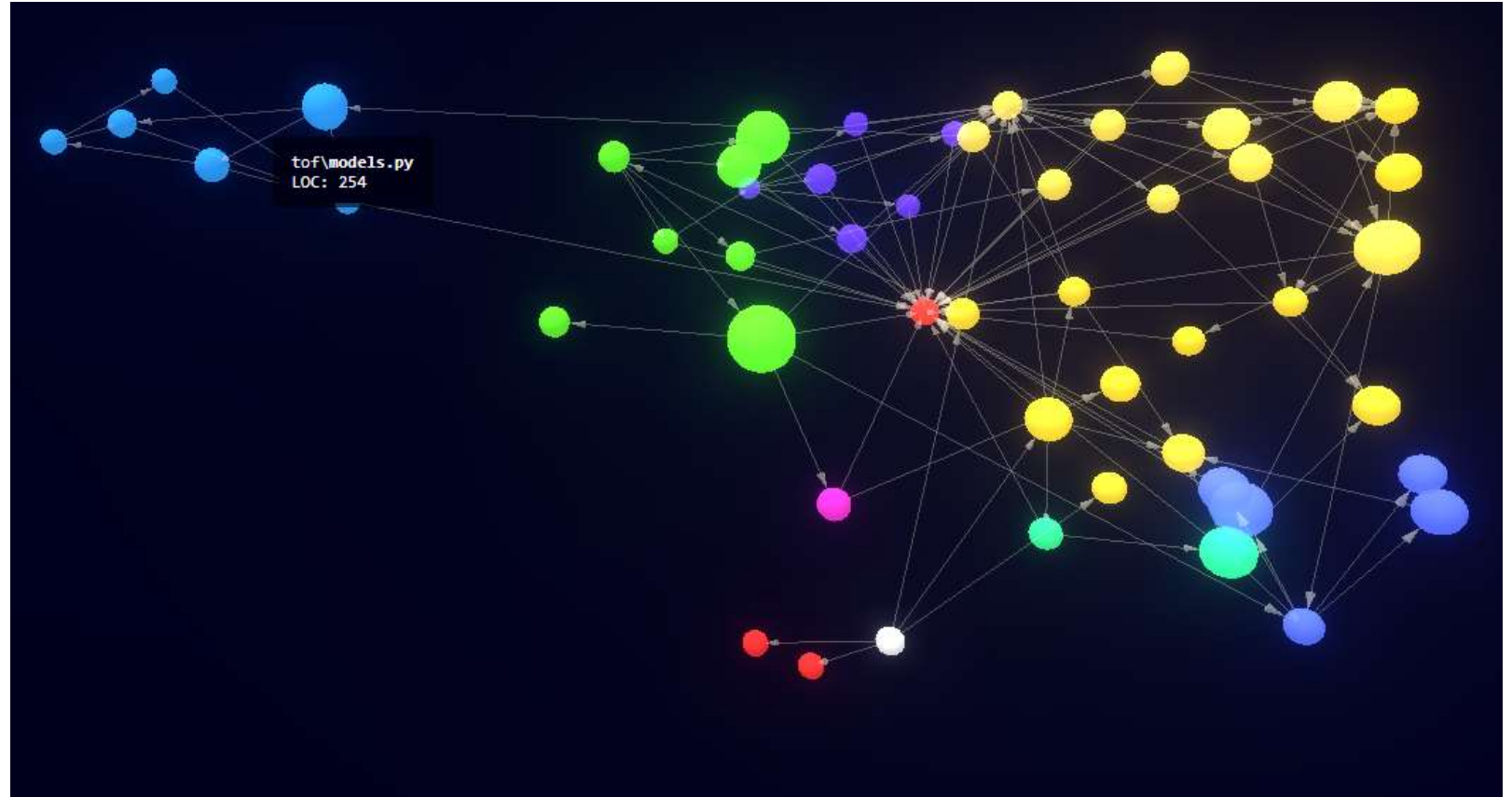
Coupling / Cohesion visualization (sc)

- Visualize the **entropy** of a code base github.com/gabotechs/dep-tree



Django Project ~270000ccl

1. App-sizes
2. Internal Cohesion
3. External Copling
4. Decoupling ways



Vortex of Doom visualization (sc)



NanoAPI visual maps
helping teams manage architecture



NanoAPI



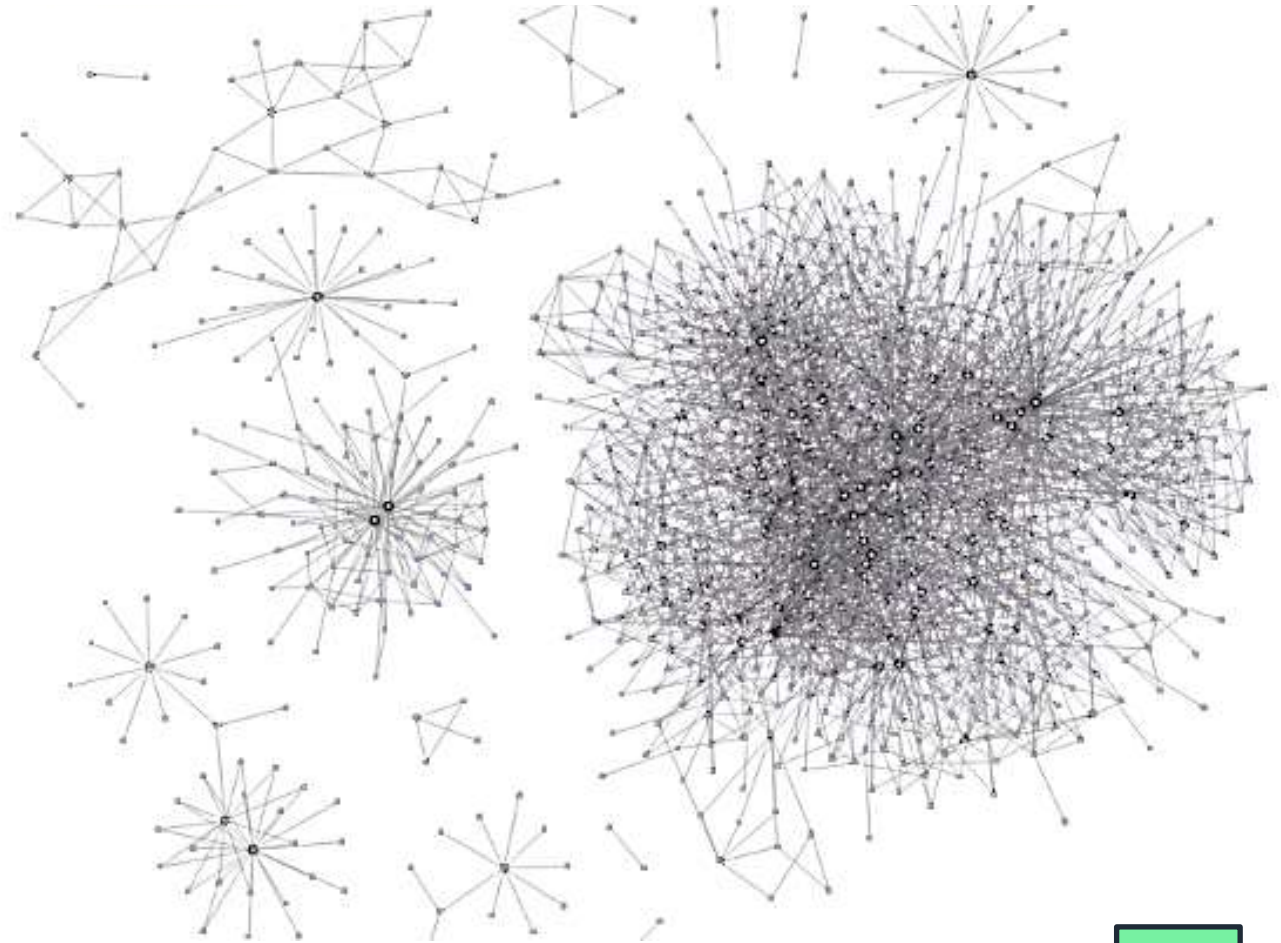
Personal



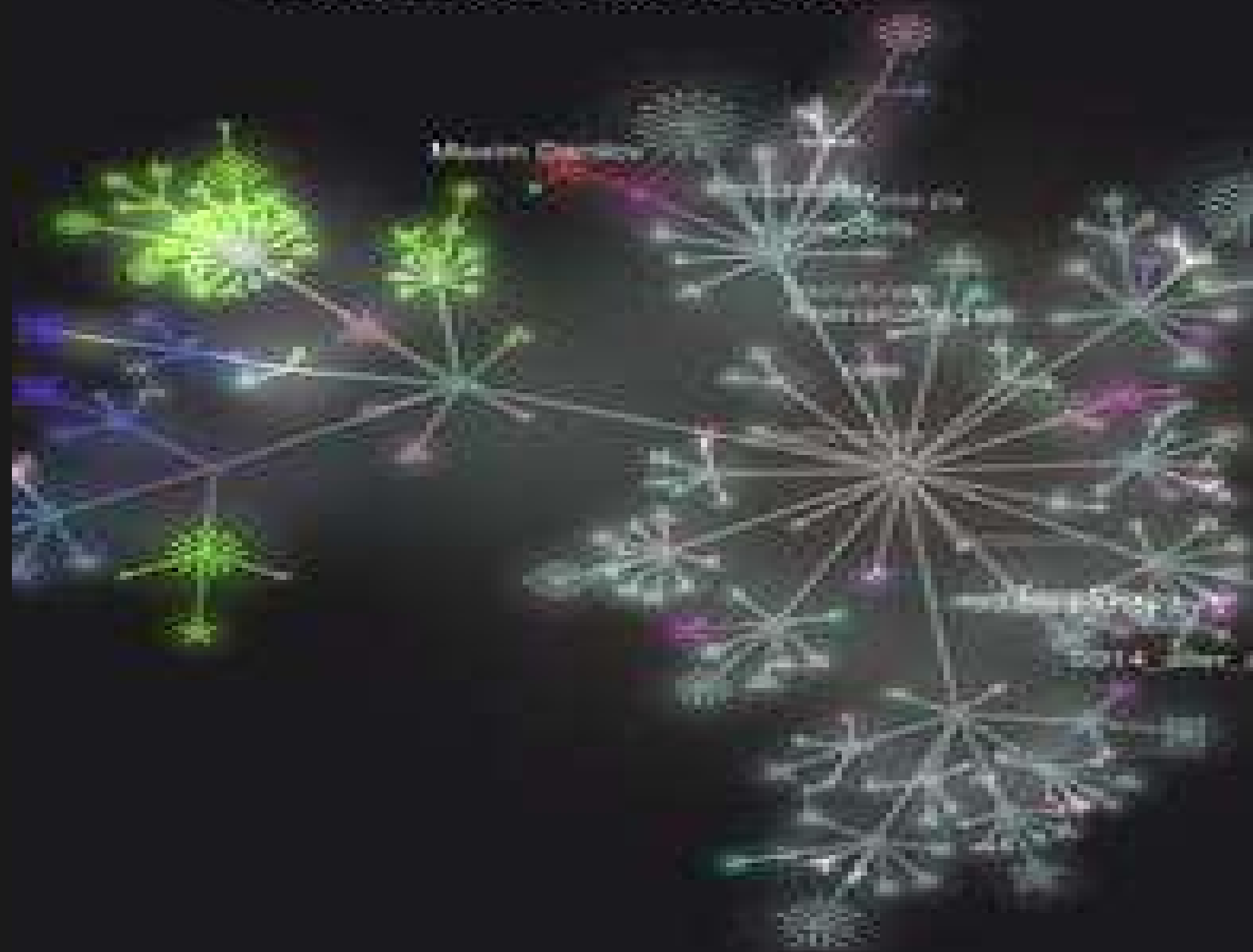
Dashboard >



Project 9 >

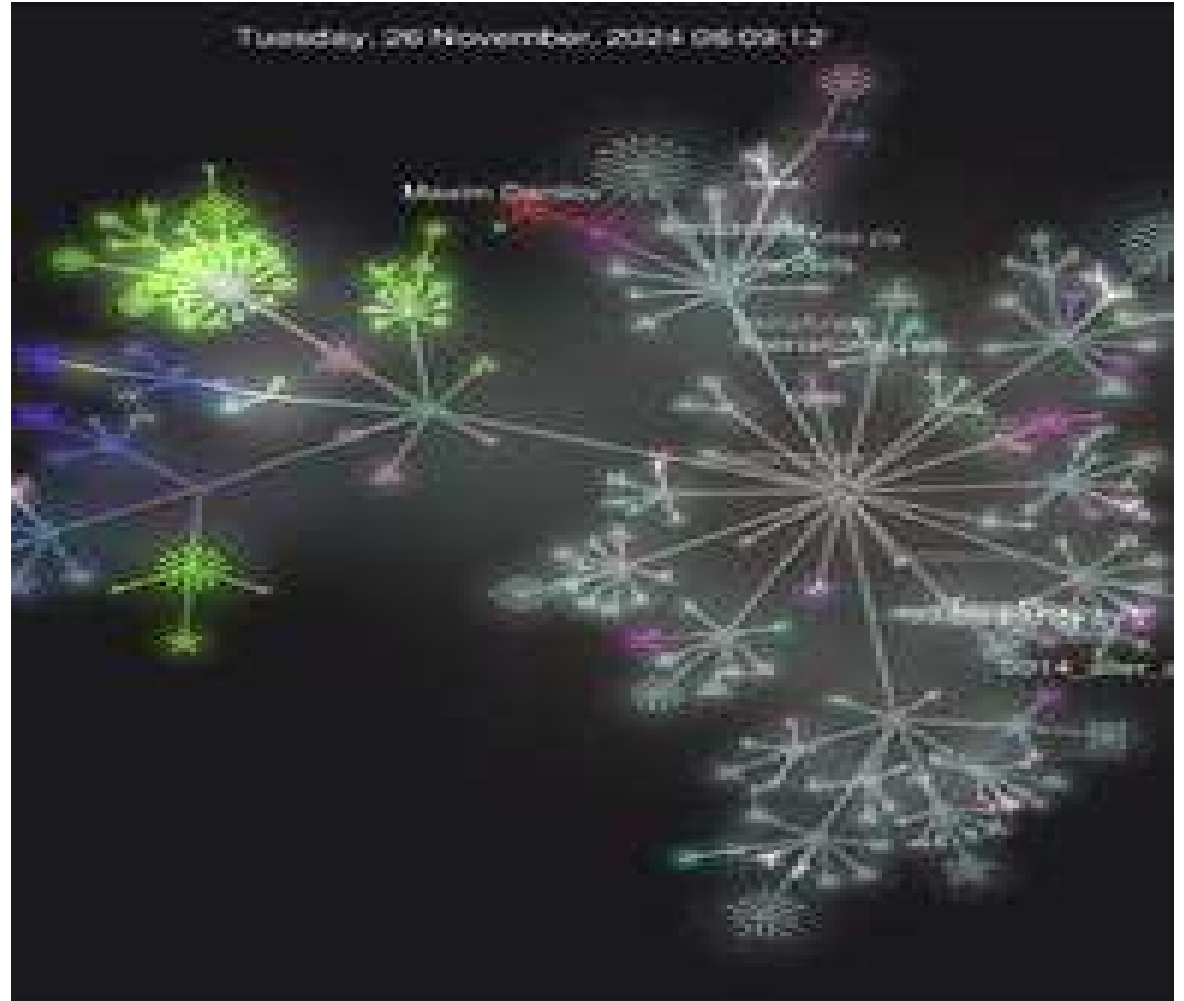


Tuesday, 26 November 2024 08:08:12



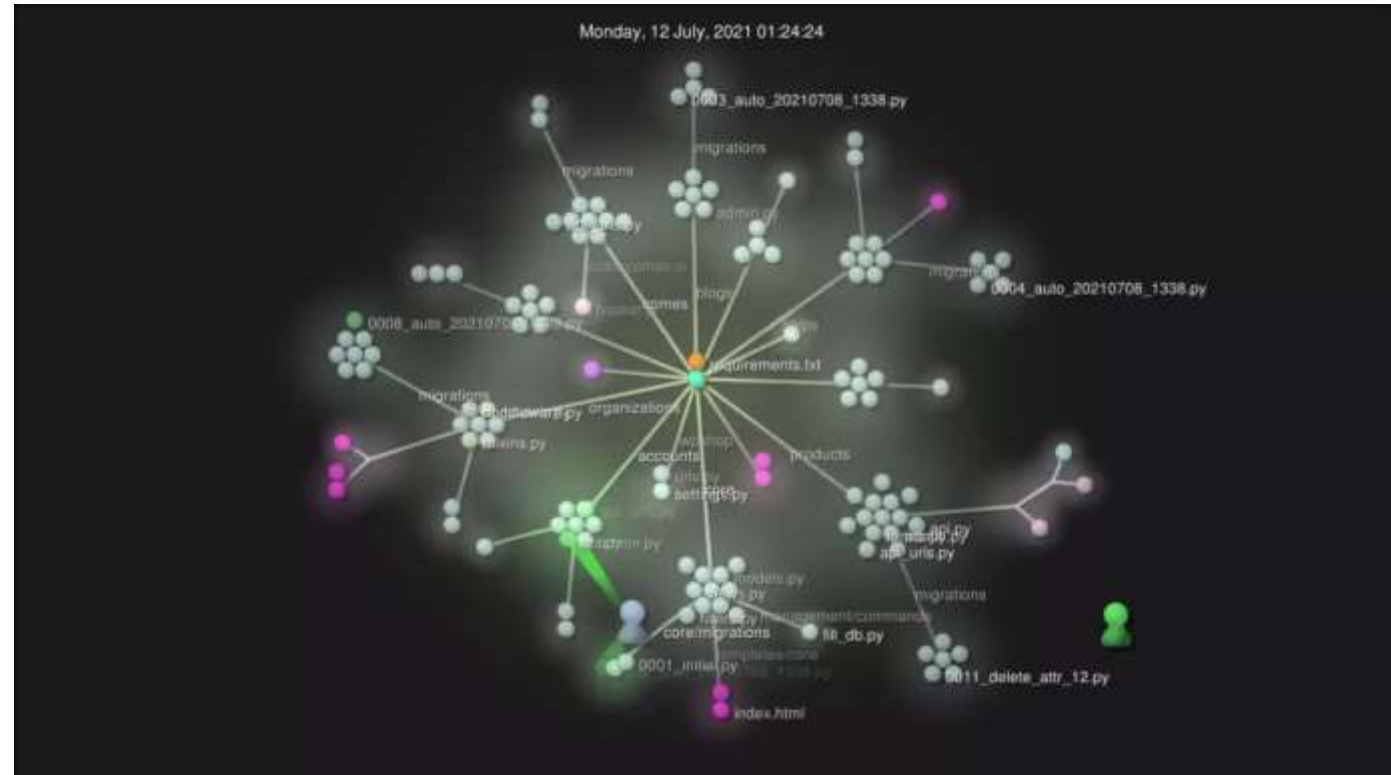
Development process visualization

1. Stability of Development
2. Matured parts
3. Modularity
4. Complexity Changing



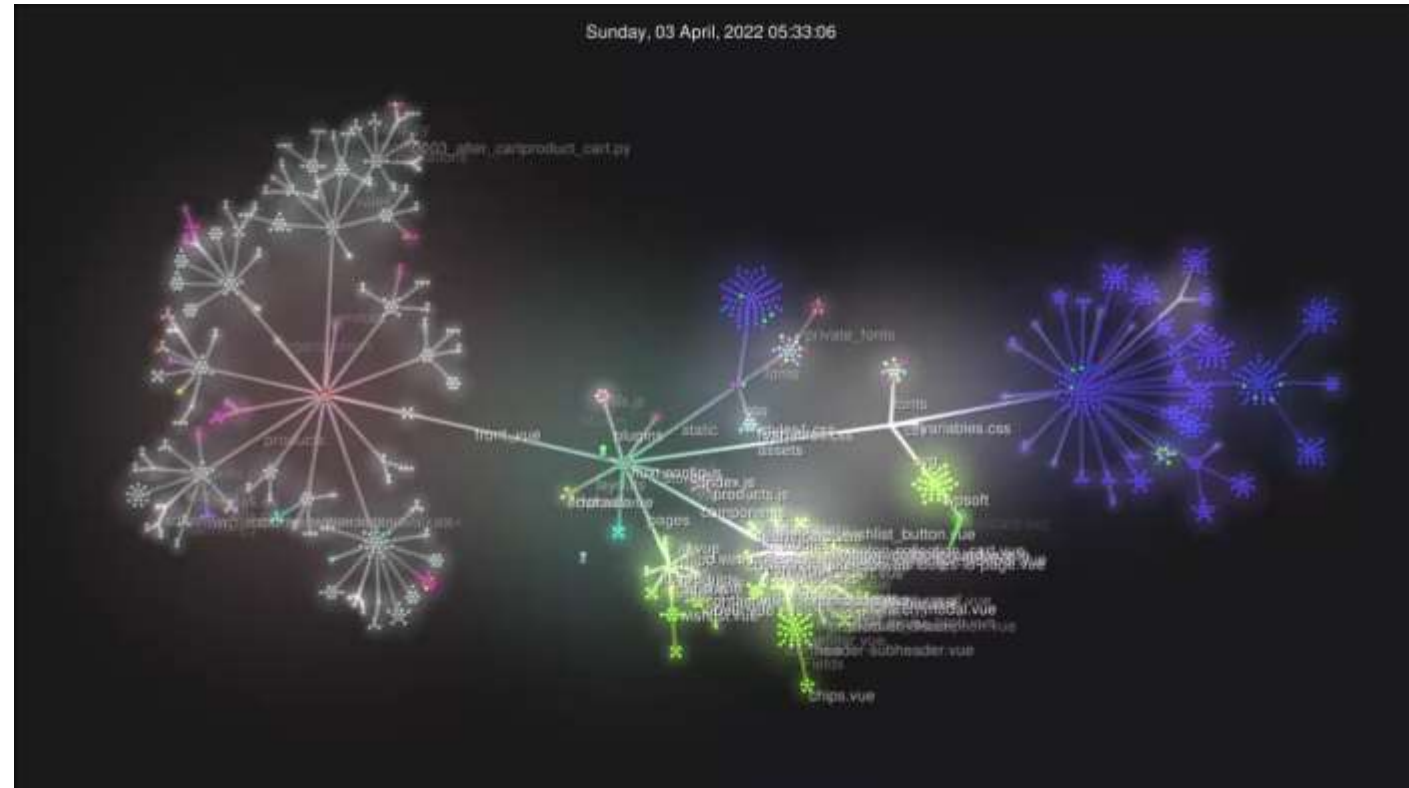
Enormous growth

1. Accidental complexity ↑
2. Structural complexity ↑
3. Lack of documentation ↑
4. Cognitive complexity ↑
5. Technical complexity – complex



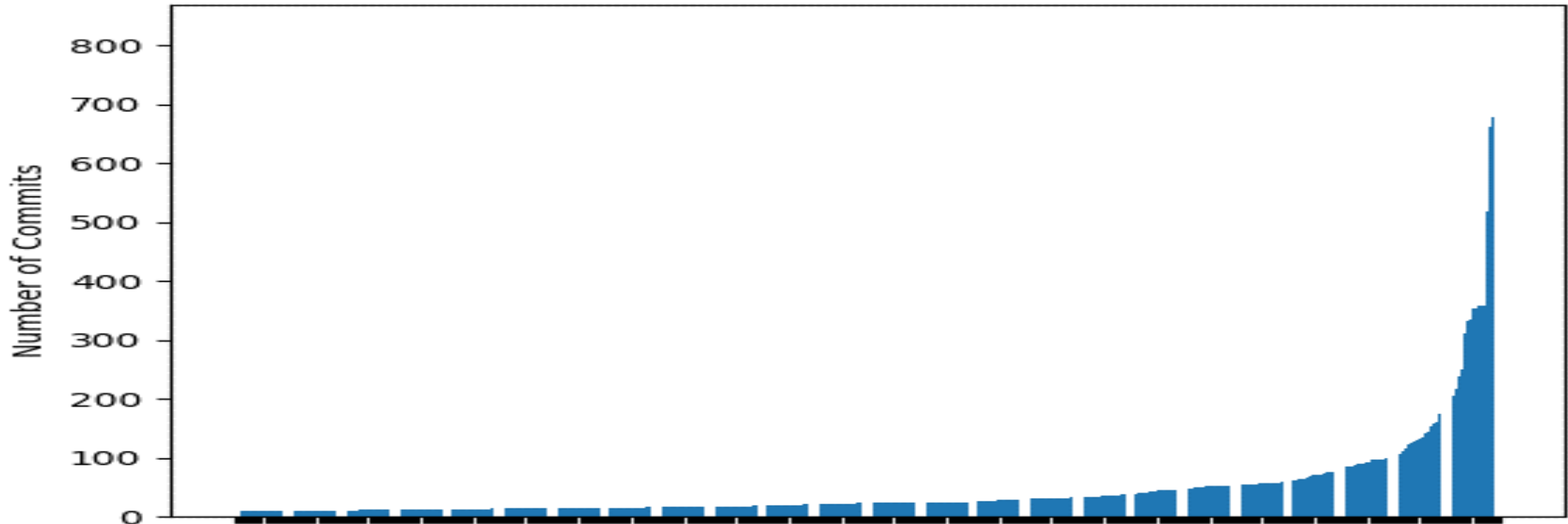
Refactoring

1. Accidental complexity ↑
2. Structural complexity ↓
3. Lack of documentation ↑
4. Cognitive complexity ↑ ↓
5. Technical complexity – still complex



Areas of high developer activity (ac)

number of commits / per file. Highest 10% of codebase



Complexity visualizing Summary

- All what you see and interpret is subjective.
- Visualization only a trick, to attract attention to problems
- Visualization is a trick to be motivated at work with complex things
- Try different tools to to find something for you



Strategies to work with complexity

Smart refactoring

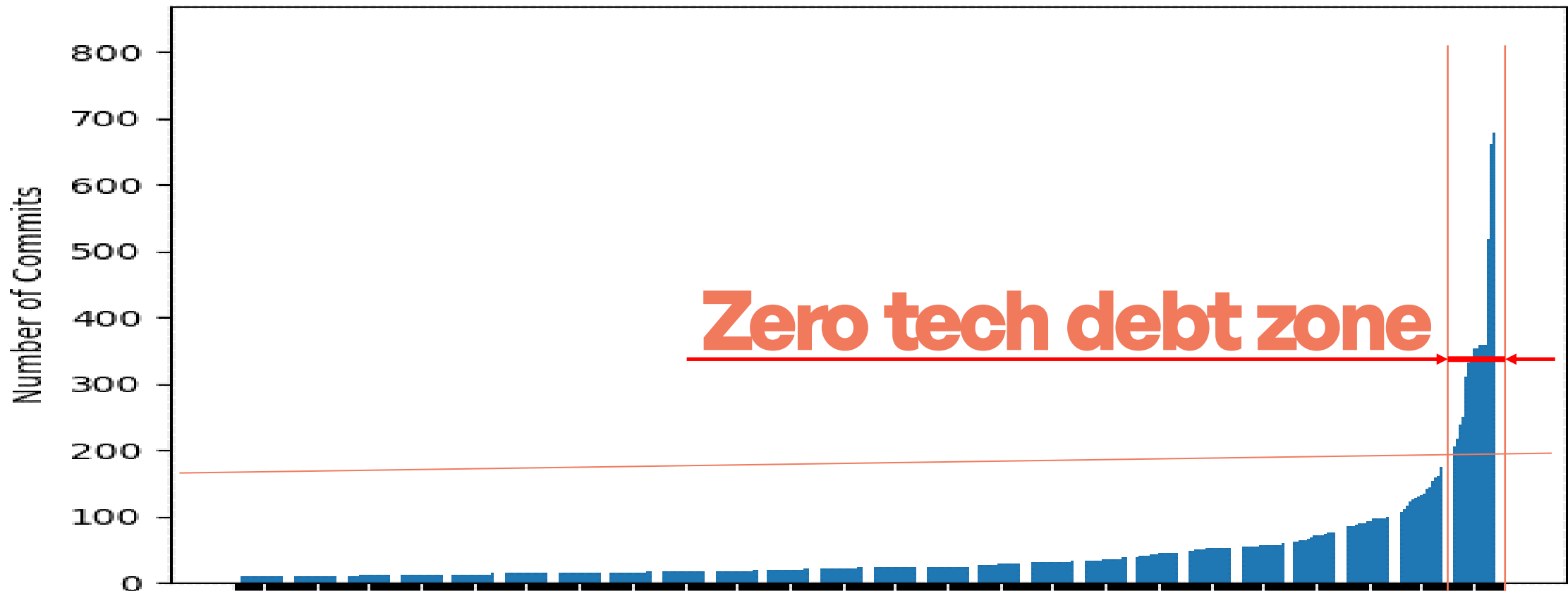
Defining refactoring Hot Zones

- Only few files change often (Michael Feathers)
- Some files must be clean from tech debt , Ignore the rest (Martin Fowler)
- 20% changes affect on 80% results (Pareto Rule)



Refactoring zone:

Feathers + Fowler + Pareto 20/80 Principle -> `fowler_feathers_pareto.py`





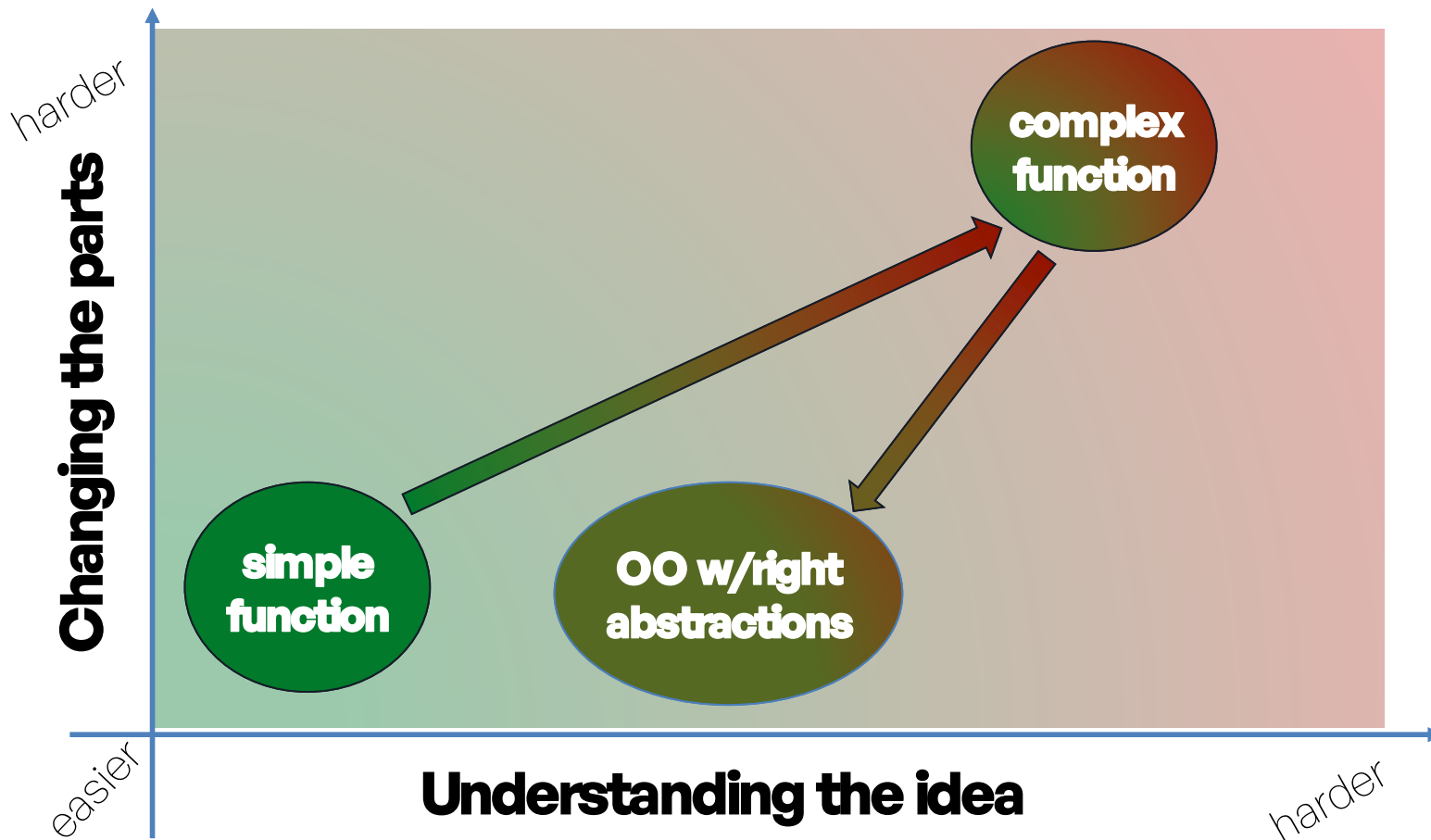
Strategies to work with complexity

Changing abstraction level

Changing abstraction level

OO solution is a bit costly than a simple function but far less costly than a complex function

- Sandi Metz



Class -> function example

DaVinci, commercial project

```
def big_func(*ag, **kw):  
    # prepare_data from *ag, **kw)  
    data = ...  
    # compute something from data  
    value = ...  
    # generate response  
    response = ...  
    return response
```



OOP

```
class MyCls:  
    def __init__(self, *ag, **kw):  
        self.prepare_data(*ag, **kw)  
  
    def compute(self, *ag, **kw):  
        ...  
        return self.post_process(values)  
  
big_func = lambda *ag, **kw: \  
    MyCls(*ag, **kw).compute()
```


A background image of a ladybug on a green leaf. The ladybug is red with black spots and is positioned in the upper left quadrant of the slide. The leaf is a vibrant green and occupies most of the background.

Strategies to distribute or reduce complexity

Increasing modularity

Modular Monolith

- 1. Organize your business logic as modular as possible*
- 2. Break it up across multiple repos*
- 3. Keep those components as a monolith*
- 4. Get reduction in complexity and a simplified management process*

Kelsey Hightower, Kubernetes

Dependency-extractor

Input:

```
>python ast_extractor.py -r project_root -f file_path
```

Output:

```
find 1894 imports  
find 24 libraries to install  
find 96 built-in dependencies  
search of dependencies finished
```

```
webhook_controller.py:
```

```
12: SyntaxWarning: invalid escape sequence '\&'
```



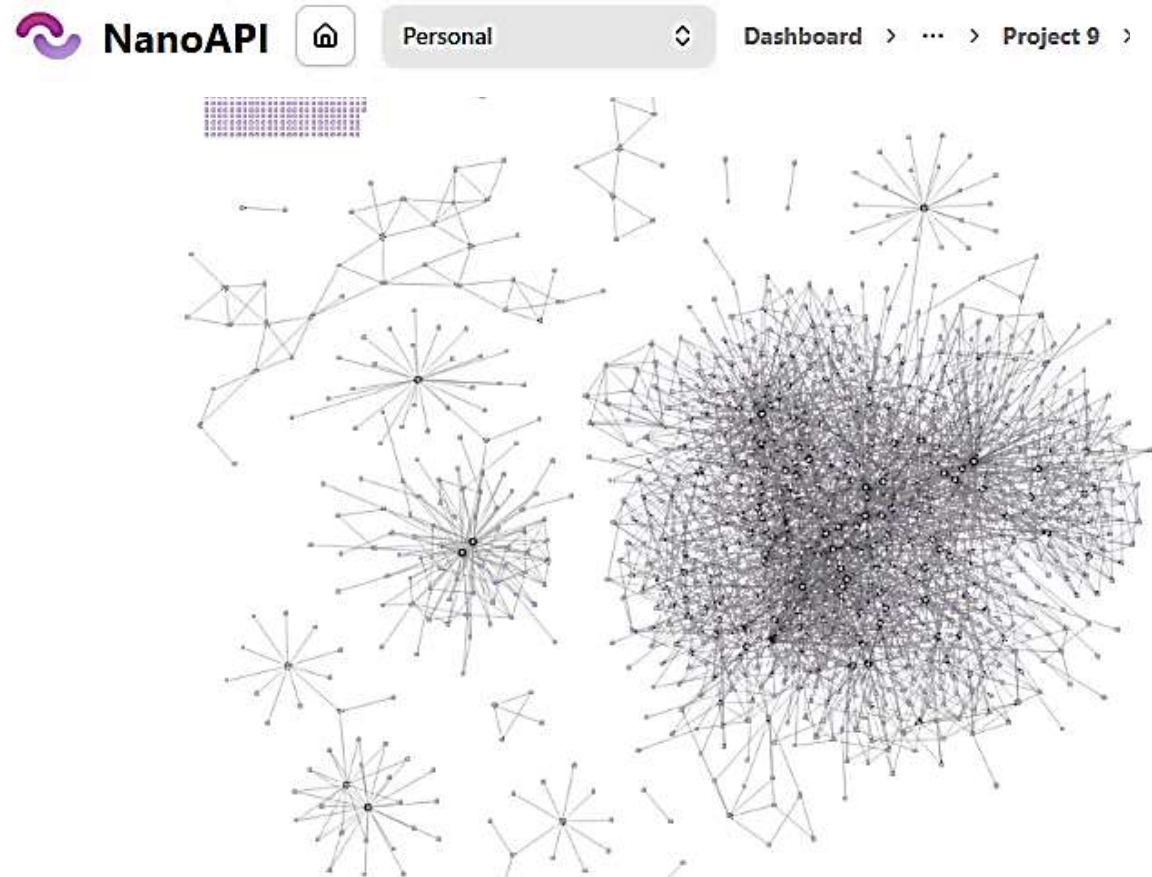
Modules extraction



NanoAPI

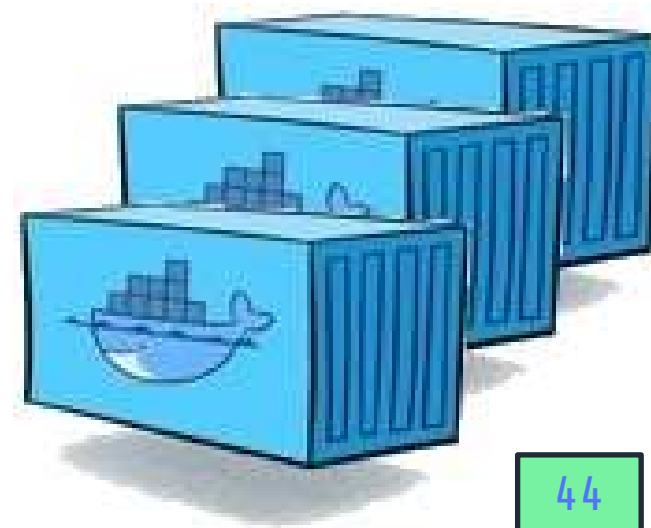
helping teams manage architecture

1. *See the structure*
2. *Extract independent modules*
3. *Die with the rest*



Isolate without refactoring

- AST-based parsing (dependency-extractor, Nano API)
- Collecting imports node (dependency-extractor)
- No runtime coupling with the rest of the codebase (dependency-extractor, Nano API)
- A working entry point with all internal dependencies.
 - Easily containerizable one or more entry points in one container.



Increasing modularity recap:

- 0% refactoring
- Extracted module can be run standalone
- Reduced container image, faster download, tests and deploy
- Extracting can be run exactly before creation of docker image



Strategies to distribute or reduce complexity

Manual Imports control

KIS

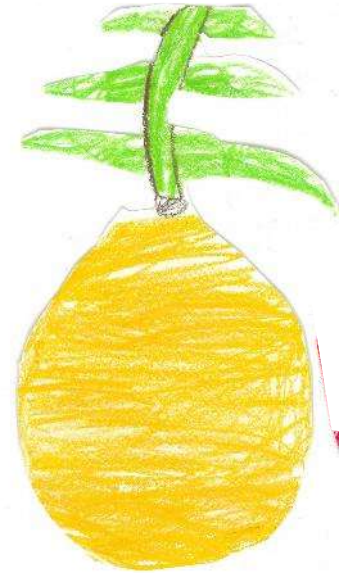
- Keep Imports Short

```
from django.db.models.fields import CharField
```

- VS

```
from django.db import models #recommended by django
```

```
from django.db.models import CharField # __all__ in __init__.py
```



TDD

Three dots deprecation

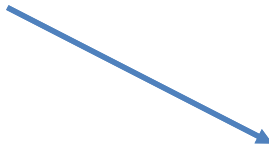
```
import api.domain_areas.observation.domain.models.observations.Observation
```

- Structural complexity ↑ ↑ ↑ ↑ ↑
- Mental load ↑ ↑ ↑ ↑ ↑
- Three dots deprecation rule in styleguide
- Linters, like **ruff** control dot-depth

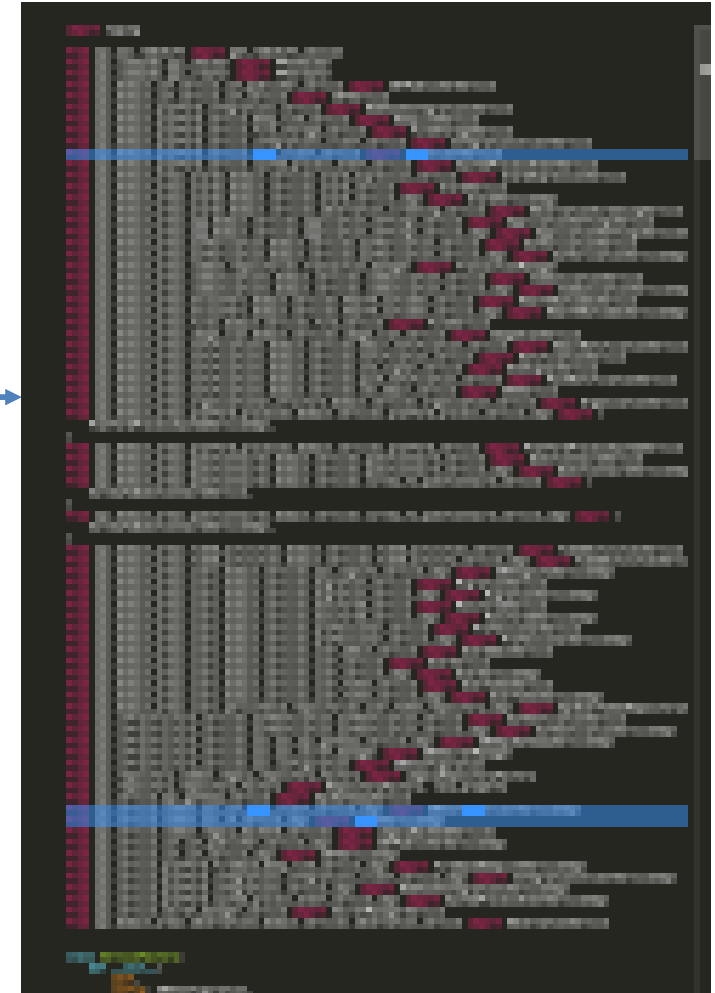


Complexity hotspots

```
module_extractor > logs > ≡ ast_modules_report.txt  
73 legacy\api\service\service_factory.py
```



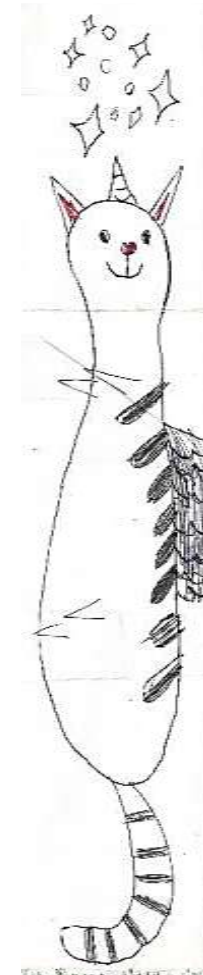
```
1 > import typing...  
81  
82
```



- 
- Coupling ↑ ↑
 - Mental load ↑ ↑ ↑

Imports control:

- Control imports – control coupling (SC)
- Long dotted Import required awareness about project
- Many imports at all increase cognitive load
- Long imports, many imports – This all increase fragility.
- Imports refactoring usually does not require testing



Strategies to change complexity

Type hints

Type Hints: Blessing or Curse?

```
from sdk import Tracker, ValidationAction
from sdk.executor import Dispatcher
from sdk.forms import FormValidationAction
from sdk.events import EventType, SlotSet
```

```
import actions.Shared.ActionServerConstants as Constants
```

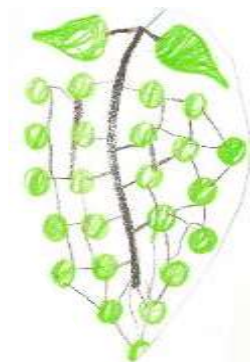
Codeium: Refactor | Codeium: Explain

```
class CustomFormValidationAction(FormValidationAction, ABC):
```

```
    ....
```

Codeium: Refactor | Explain | Generate Docstring | X

```
    .... async def run(self, dispatcher: Dispatcher, tracker: Tracker, domain: Dict[Text, Any],) -> List[EventType]:
    ....     events = []
    ....     if not tracker.get_slot("slot_form_initialized"):
    ....         events.extend(await self.extra_run_logic(dispatcher, tracker, domain))
    ....     events.extend(await super().run(dispatcher, tracker, domain))
    ....     return events
```



What if



What if we replace all type hints with “typing.Any”,
then remove unused imports??

Input:

```
>python perform_extraction.py -r project_root -f path_to_file  
>autoflake --remove-all-unused-imports --inplace
```

Coupling with and without type hints:

A project with type hints

- find 560 dependencies
- find 21 libraries to install
- find 53 built-in dependencies
- summary files with hints: 414
- summary lines of code: 39638
- summary type hints used: 4153
- Finally package: 787 files in 234 folders, 1,46 MB.

The same project without type hints

~18% less

- find 505 dependencies
- find 20 libraries to install
- find 52 built-in dependencies
- summary files with hints: 362
- summary lines of code: 29901
- summary type hints used: 4075
- Finally package: 731 files in 233 folders, 1,19 MB.



Alternatives for Type-Hints

- Type-hints in stub files (*.pyi , PEP 484)
 - + 0% refactoring
 - + generation, update, check can be automated
 - + full ide and linters support
 - + don't create noise in code file
- Type-hints in docstrings
 - + ready for autodocumentation in any style: GDOC, SPHINX, ReDOC...
 - + generation, update, check can be automated
 - + full ide and linters support
 - + collapsable, may be hidden in ide



If Type Hints Are Required:

- Type-hints required in some cases:
 - Dataclasses
 - libraries, where type hints used for validation or code generation.

Add type-hints on production from Stubs (.pyi)
or from Docstrings with **@typingdecorator**



@typingdecorator

```
@typingdecorator
class Example:
    """ Example dataclass.
    Attributes
    -----
    foo : int
        The foo attribute. Default is 123.
    bar : str, optional
        The bar attribute."""

    def info(self):
        """
        :returns: str: Some info
        """
        return f"foo={self.foo}, bar={self.bar}"
```



```
# Test
>> example = Example(foo=5, bar="xyz")

>> print(example)
Example(foo=5, bar='xyz')

>> print(example.info.__annotations__
{'return': <class 'str'>})

>> print(Example.__annotations__)
{'foo': <class 'int'>,
'bar': typing.Optional[str]}
```

Type Hints recap:

- Type Hints increase cognitive and structural complexity
- Type Hints increase weight of package, and codelines count
- Stub file keep information about types, 0% refactoring in typeless projects
- doc strings can keep information about types
- @typingdecorator cover cases, if type-hints are required

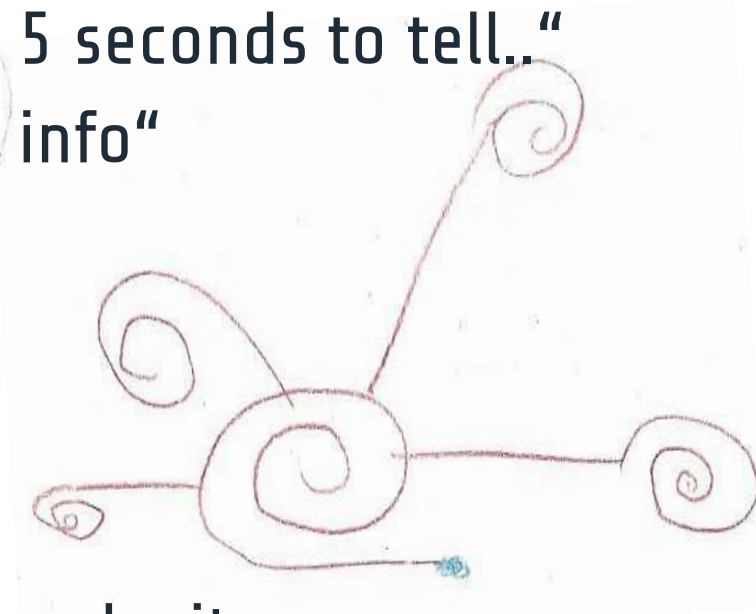


Strategies to distribute or reduce complexity

documentation

Documenting:

1. Write documentation in style „if I die, everything should work“:
 - only unknown information in style „i have only last 5 seconds to tell..“
 - add security data in form „where i can get required info“
2. Allow LLM add known information, if it is important.
Other case skip it.
3. Doc – string coverage can help to reduce cognitive complexity



Final words

recup

Summary :

1. Work with complexity lead to better design.
2. Measure complexity to be ready to change it.
3. Visualize complexity to be ready to change it.
4. If you are ready – Smart refactoring is an opportunity to change complexity.
5. Adding type hints is an opportunity to increase complexity and coupling.
6. Frequently imported internal dependencies: Extract. Isolate. Simplify
7. Use tools like nanoAPI to isolate selected dependencies – it works “fast and furious”.



Try it yourself

- Install Gource from gource.io/ or napi-cli from nanoapi.io/
- Clone locally any long living github python project
- Run in Cli `gource` or `napi init`
- Enjoy



Obrigado Cascais and you for attention!



Maxim Danilov
maxim@wpshop.at



Play with code: github.com/danilovmy/pyconpt2025

Useful links

- <https://martinfowler.com/bliki/TechnicalDebt.html>
- <https://www.agileconnection.com/article/getting-empirical-about-refactoring>
- <https://www.forbes.com/councils/forbestechcouncil/2023/09/14/taking-a-more-strategic-approach-to-the-8020-rule-in-software/>
- <https://sandimetz.com/blog/2017/9/13/breaking-up-the-behemoth>
- <https://x.com/kelseyhightower/status/1621184564956893189>

Easter Egg Slide

- If complexity had a face...

```
def timing[**P, R] (f: Callable[P, R]) -> Callable[P, R]:  
    def inner(*args: P.args, **kwargs: P.kwargs) -> R:  
        t = time.time()  
        retval = f(*args, **kwargs)  
        print(f"{f} took {time.time() - t} seconds")  
    return retval  
return inner
```

Easter Egg Slide

- If complexity had a face...

ule_extractor > legacy > api > infrastructure_services > aws >  aws_client_service.py

```
class AWSClientService:
    ... def __init__(self, aws_access_key_id: int, aws_secret_access_key: str, aws_region_name: str):
    ...     self._aws_access_key_id = int(aws_access_key_id)
    ...     self._aws_secret_access_key = aws_secret_access_key
    ...     self._aws_region_name = aws_region_name
    ...     self._aws_session = None
```